

COMP 424 – Homework 1

Muhammad Armaghan Rashid · Student ID: _____ · Fall 2026

Question 3: Search for optimization

Setup. Hill climbing moves to the best of the (up to 8) neighbours in `MOVES` and stops when none is strictly better than `max_value`; `num_steps` = number of moves. Local beam search keeps the `k` best coordinates out of all neighbours (`options`) of the current `k` coordinates and stops the same way; with `k = 1` it is exactly hill climbing, so my code uses one function, `climb(function, coordinates, step)`, for both (`k = len(coordinates)`, set by `width`). `new_x` and `new_y` must stay in $[0, 10]^2$. I reset the random seed (`np.random.seed(5)`) for each setting, so for a given `width` every step gets the same 100 starts. “Hit” = the run ended at the global max (`max_value > threshold`, with `threshold = 1.95` for `f1` and `9` for `f2`). The global max of `f1` is 2 at $(\pi/4 + j\pi, 0)$, i.e. $x \approx 0.79, 3.93, 7.07$ with $y = 0$. For $y \geq 0$, $0.5y + 1 > 0$, so $f2 = |x - 2| + 0.5y - 3$: it grows with $|x - 2|$ and with y , so its global max is at the far corner $(10, 10)$: $8 + 5 - 3 = 10$. The other corner at $y = 10$ is a local max: $f2(0, 10) = 2 + 5 - 3 = 4$. (A grid search over $[0, 10]^2$ gives the same two points.) The thresholds split the runs cleanly: in hill climbing every miss ended at 1.912 or less (`f1`) and 3.998 or less (`f2`), and every hit at 1.980 or more (`f1`) and 9.706 or more (`f2`).

In the tables, `steps` = `num_steps` and `f*` = `max_value` returned by `climb`, averaged over the 100 runs (`steps_list, coordinate_list`); `sd` = standard deviation (`np.std`).

(a) Hill climbing (`width = 1`, 100 random starts per step)

step	f1					f2				
	steps mean	steps sd	f1* mean	f1* sd	hits	steps mean	steps sd	f2* mean	f2* sd	hits
0.01	272.6	160.8	1.729	0.351	58	572.3	235.3	8.853	2.354	81
0.05	54.2	32.1	1.723	0.357	58	114.1	47.1	8.824	2.352	81
0.1	26.9	16.0	1.713	0.367	58	56.8	23.5	8.788	2.347	81
0.2	13.2	8.0	1.689	0.386	58	28.1	11.7	8.713	2.347	81

- `num_steps` $\propto 1/\text{step}$: `f1` $273 \rightarrow 54 \rightarrow 27 \rightarrow 13$ and `f2` $572 \rightarrow 114 \rightarrow 57 \rightarrow 28$; the `sd` shrinks the same way. `num_steps` $\times$ `step` stays ≈ 2.7 (`f1`) and ≈ 5.7 (`f2`): that is the distance walked, which does not depend on the step.
- **The step does not change where you end up:** 58/100 (`f1`) and 81/100 (`f2`) hits at every step. Hill climbing just climbs the hill it starts on. `f1` gets stuck on the edge $y = 10$ if $y > 2\pi$ (≈ 1.28 at $x = \pi/4 + j\pi$, or ≈ 1.20 at the corner $(10, 10)$), or at $(10, 0)$ (≈ 1.91) if $x > 3\pi/4 + 2\pi \approx 8.64$; exactly the 42 starts in these regions miss. `f2` gets stuck at $(0, 10)$ (value 4) if $x < 2$; exactly the 19 starts with $x < 2$ miss, and all 19 end at $(0, 10)$. These local maxima are why the `sd` of `f*` is so big.
- **Bigger steps end a bit lower** (`f1` $1.729 \rightarrow 1.689$, `f2` $8.853 \rightarrow 8.713$) because the search stops up to one step away from the real top.

(b) Local beam search ($k = \text{width}$, 100 runs per setting; step 0.1 shown, all steps in the output)

k	f1					f2				
	steps mean	steps sd	f1* mean	f1* sd	hits	steps mean	steps sd	f2* mean	f2* sd	hits
1 (hill)	26.9	16.0	1.713	0.367	58	56.8	23.5	8.788	2.347	81
2	22.6	14.4	1.735	0.352	60	48.4	24.1	9.325	1.799	90
4	18.5	13.6	1.809	0.316	69	43.3	22.1	9.691	1.174	96
8	14.7	10.2	1.924	0.210	84	32.4	16.6	9.930	0.027	100
16	10.3	6.6	1.969	0.128	91	25.5	14.6	9.927	0.030	100

- **Yes, it improves on hill climbing for both functions.** Final value: at step 0.1, f1 hits go 58 → 60 → 69 → 84 → 91 for $k = 2, 4, 8, 16$ and the f1* sd drops from 0.367 to 0.128; for f2 they go 81 → 90 → 96 → 100 → 100 and the sd drops from 2.347 to 0.030.
- **Iterations:** the mean of `num_steps` drops as k grows at every step, and for $k \geq 4$ the sd drops too at every step (at $k = 2$ it is mixed: lower for f1, e.g. 16.0 → 14.4, but slightly higher for f2, 23.5 → 24.1): at $k = 16$, f1 26.9 → 10.3 (sd 16.0 → 6.6) and f2 56.8 → 25.5 (sd 23.5 → 14.6).
- **Why:** it starts from k random `coordinates` and keeps the best successors of all of them, so it carries on from whichever start is best placed.
- **But:** each iteration evaluates up to $8k$ neighbours instead of 8, so fewer iterations still cost more: counting calls to `function(f1, step 0.1)`, a run takes 266 evaluations on average for hill climbing vs 1,781 for $k = 16$. For f2, $k = 8$ already gets 100/100 at every step, so $k = 16$ only adds cost. And it can still miss (9/100 for f1 with $k = 16$; 6 end at (10, 0), 3 on the edge $y = 10$). With 16 starts, at least one almost always starts on a global-max hill, so these misses come from the selection: the beam keeps only the k best neighbours overall, and points climbing a local hill can be higher early on and push out the lower points that were on a global-max hill.

Q3 output

```
Results for f1
step    width  steps mean  steps sd    f* mean  f* sd    hits
0.01     1    272.6    160.8    1.729    0.351    58/100
0.01     2    226.6    138.3    1.732    0.347    58/100
0.01     4    190.1    138.0    1.791    0.323    65/100
0.01     8    149.6    108.4    1.880    0.262    78/100
0.01    16    117.2     73.1    1.956    0.157    86/100
0.05     1     54.2     32.1    1.723    0.357    58/100
0.05     2     44.9     27.6    1.734    0.349    59/100
0.05     4     38.0     27.8    1.808    0.316    68/100
0.05     8     29.4     21.3    1.898    0.243    80/100
0.05    16     21.3     12.9    1.955    0.159    88/100
0.1      1     26.9     16.0    1.713    0.367    58/100
0.1      2     22.6     14.4    1.735    0.352    60/100
0.1      4     18.5     13.6    1.809    0.316    69/100
0.1      8     14.7     10.2    1.924    0.210    84/100
0.1     16     10.3      6.6    1.969    0.128    91/100
0.2      1     13.2      8.0    1.689    0.386    58/100
0.2      2     11.0      7.1    1.717    0.364    60/100
0.2      4      8.8      6.7    1.797    0.325    69/100
0.2      8      7.0      4.7    1.927    0.207    89/100
0.2     16      5.0      3.2    1.980    0.081    96/100

Results for f2
step    width  steps mean  steps sd    f* mean  f* sd    hits
0.01     1    572.3    235.3    8.853    2.354    81/100
0.01     2    487.8    240.7    9.392    1.800    90/100
0.01     4    433.7    220.8    9.692    1.308    95/100
0.01     8    334.3    171.8    9.993    0.003   100/100
0.01    16    266.7    155.9    9.992    0.003   100/100
0.05     1    114.1     47.1    8.824    2.352    81/100
0.05     2     97.2     48.2    9.362    1.798    90/100
0.05     4     86.1     44.0    9.662    1.306    95/100
0.05     8     66.5     34.3    9.961    0.015   100/100
0.05    16     52.9     31.4    9.963    0.015   100/100
0.1      1     56.8     23.5    8.788    2.347    81/100
0.1      2     48.4     24.1    9.325    1.799    90/100
0.1      4     43.3     22.1    9.691    1.174    96/100
0.1      8     32.4     16.6    9.930    0.027   100/100
0.1     16     25.5     14.6    9.927    0.030   100/100
0.2      1     28.1     11.7    8.713    2.347    81/100
0.2      2     23.9     12.0    9.254    1.797    90/100
0.2      4     21.0     10.7    9.617    1.173    96/100
0.2      8     15.9      8.1    9.860    0.062   100/100
0.2     16     11.9      6.4    9.852    0.062   100/100
```